

NCSA REGIONAL WORKSHOP ON AI · TRACK 2

Getting started using AI agents for research and HPC

Today you work with a coding agent on Delta: first a prepared project, then a problem of your own.

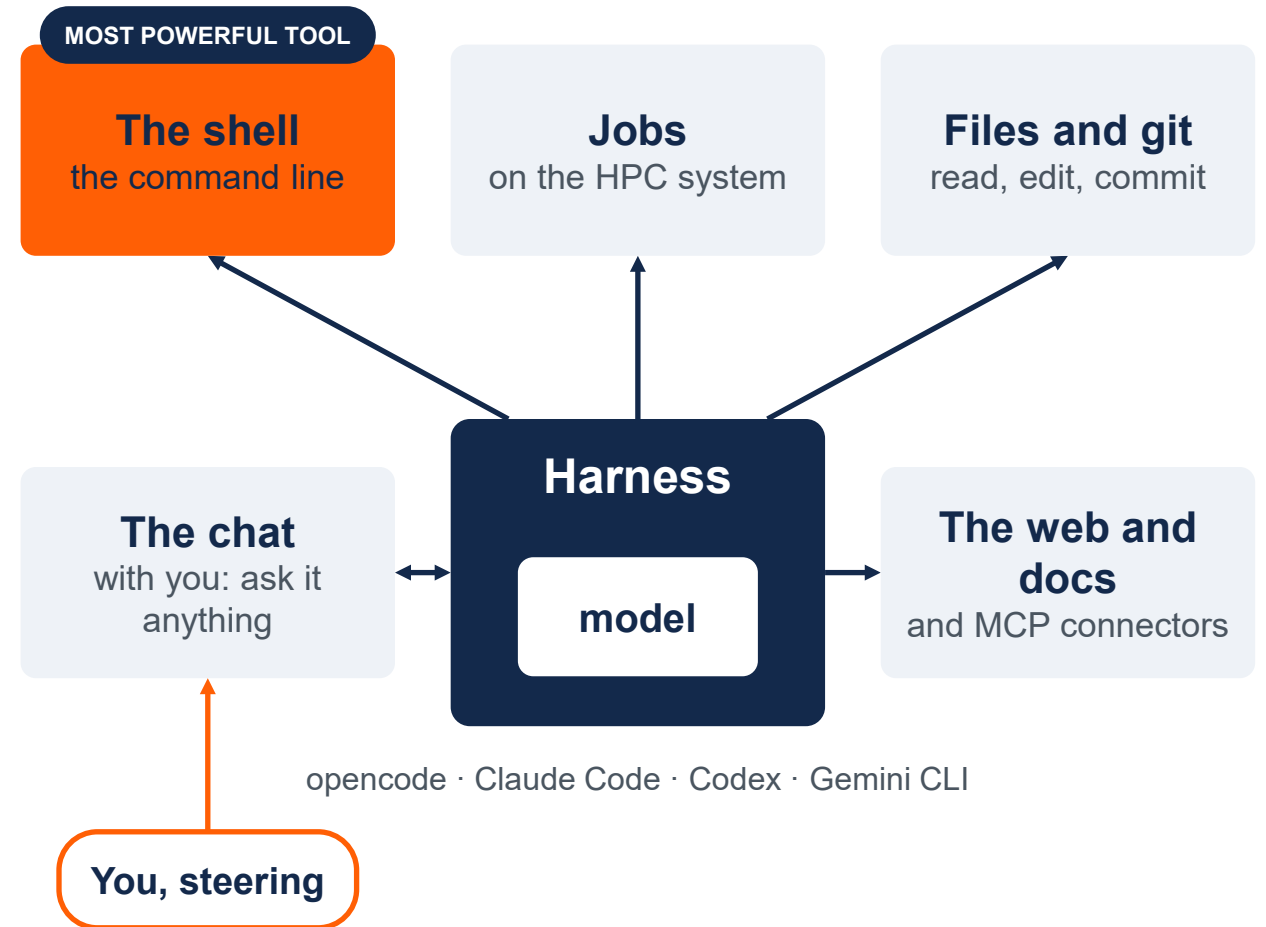
You leave with two real jobs run and checked, and a guide for doing this on your own projects.

Mon Oct 5 2026 · 10:00–11:45 · Room 1040

Facilitators: Darren Adams, Soham Pal (NCSA)

An agentic coding harness uses models and tools

- A **harness** gives the model tools and runs a loop: read, plan, act, check.
- You ask in plain language; it reads files, runs commands, edits code, uses context and scaffolding (rules, guardrails goals, etc.) and shows you what happened.
- **The command line is its supepower:** whatever you can do in a terminal, it can do, at your direction.
- You steer: it asks before anything consequential.



Common Coding Harnesses

Commercial

from a vendor, used with its plan or key

Claude Code

Anthropic · Claude models
terminal · editor

Codex

OpenAI · GPT models
terminal · editor

Gemini CLI

Google · Gemini models
terminal

Copilot CLI

GitHub · Claude, GPT and
Gemini models
terminal · editor

Open source

community projects; you choose the model

opencode

75+ model providers, including local models
terminal · editor

On NCSA systems: hpc-gpt

opencode with the NCSA Lumen models

Aider

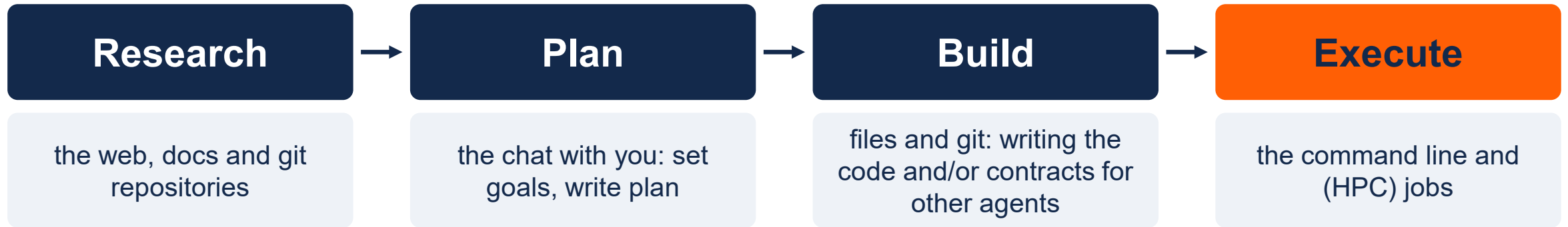
Goose

Cline

Editor-first tools: Cursor · Google Antigravity · Copilot in VS Code

Any of them can reach an HPC system: a terminal over ssh; an editor through VS Code Remote-SSH, or Code Server in Open OnDemand.

Beyond the shell: research, plan, build, execute



- More tools: the web, git repositories, documentation, and other services through connectors (MCP servers).
- Today's agent can query the Delta documentation and the scheduler.
- The chat itself: ask it a question. Why did that job wait? What does this flag do?

Scaffolding: what you build around an agent

Scaffolding is the structure around the agent that keeps its work on track, and lets you check it.

Instructions

what the agent reads first: how to work, what never to do

A plan

the goal, milestones with evidence, your typed approval

Evidence

create, then verify against a well-defined outcome. Exit 0 is not success.

Permissions

some commands run without asking, some ask you first, some never run

```
[ws-env] host=cn003.delta.ncsa.illinois.edu job=22639277 gpu=none module=none
PROGRAM: sv_sim N: 24 GATES: 200
AMP_ERR: 3.253e-19
RESULT: 1.000000000000 TIME: 7.2372 WORKERS: 1
PROGRAM: sv_sim N: 24 GATES: 200
AMP_ERR: 3.253e-19
RESULT: 1.000000000000 TIME: 2.1093 WORKERS: 16
PROB_SUM: 1.000000000000
SPEEDUP_SV16: 3.43
ELAPSED_S: 12
RESULTS_OK
```

Evidence, from a real job log: where it ran, **KEY: value** lines, and **RESULTS_OK** only on success.

Scaffolding for the workshop

House rules

AGENTS.md

what every harness reads first: how to work, what never to do, and why

Session guide

guide/

one short script per module, read by the agent as you go

Helpers

ws-*

ws-submit is sbatch with the workshop's checks; it shows the exact sbatch line it runs

Your plan and the agent's memory

PLAN.md, **PROGRESS.md**

```
$ ws-submit topics/ml-gpu/job.sbatch
[ws] ws-submit runs this sbatch command (dry run: nothing submitted yet):
sbatch --account=bccu-delta-gpu --partition=gpuA40x4 --time=00:10:00 --job-name=ws-ml-gpu
      --nodes=1 --ntasks-per-node=1 --cpus-per-task=16 --gpus-per-node=1 --mem=48g
      --output=slurm-%j.out --error=slurm-%j.out --comment=ws:ml-gpu topics/ml-gpu/job.sbatch
[ws] notice: the workshop reservation sup-30882 is magnetic: it takes this job by itself when it
      has room (no --reservation needed), else the job runs outside it in the general queue
DRY RUN: nothing submitted. Rerun with --yes; the harness asks the participant.
```

```
$ sacct -P -j 22639277 -o JobID,State,ExitCode,Elapsed,NodeList,Partition,AllocTRES
JobID|State|ExitCode|Elapsed|NodeList|Partition|AllocTRES
22639277|COMPLETED|0:0|00:00:19|cn003|cpu|billing=16000,cpu=16,mem=16G,node=1
```

ws-submit is **sbatch** with the workshop's checks. It shows the command it runs (one line in the terminal; wrapped here).

The scheduler's record: it ran where we think. Completed, on a CPU compute node, with 16 cores.

Special to a shared workshop: **ON-YOUR-OWN.md** shows the short version you would write yourself.

Prepared Projects

Pick a goal → the agent sets up open/ as a small project → runs a job → you steer by typing

Fit a scikit-learn model

ridge, then a random forest;
report the test error

Speed up a Python loop with numba

time both versions

CPU vs GPU matrix multiply

at three sizes

Summarise a public CSV

with one seaborn plot

Monte Carlo π in C with OpenMP

time 1 and 16 threads

Summarise ten paragraphs with a local LLM

through LLMFlux

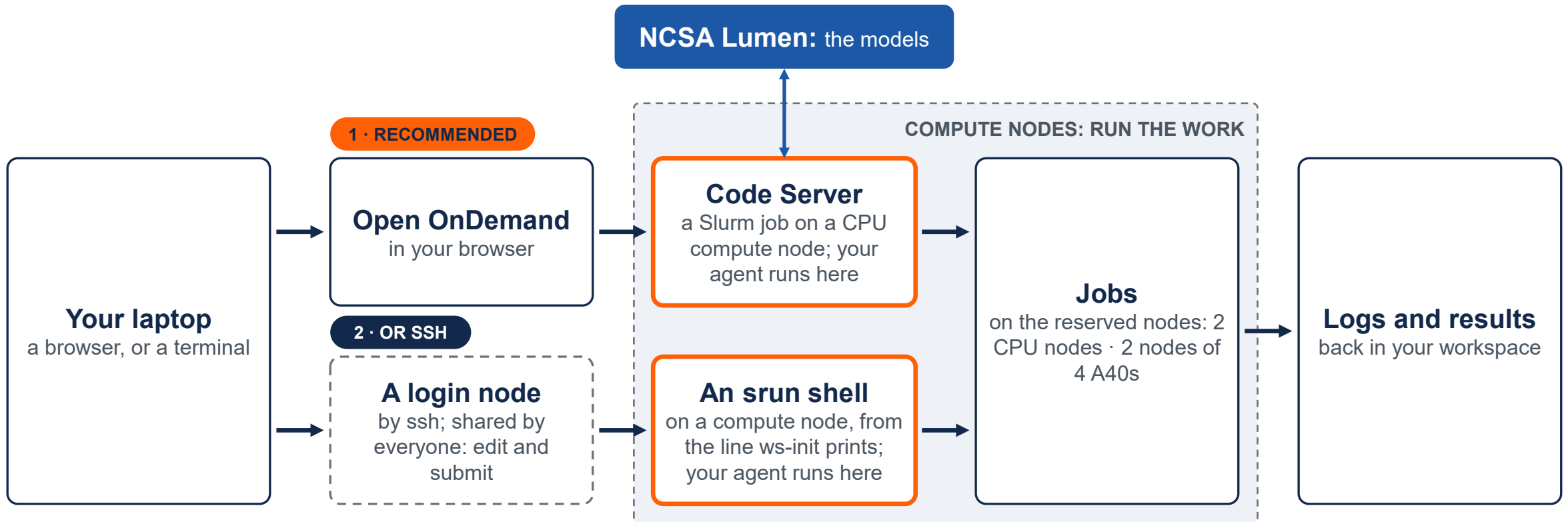
The Lorenz system with SciPy

how fast two nearby starts
separate

Or your own

Limits: each job at most 1 GPU or 16 cores, and 15 minutes · **ten open-ended jobs in all**, each answering one question · a CPU job unless the step needs the GPU.

NCSA Delta System: Logging in



- **Open OnDemand → Code Server:** your session is itself a Slurm job on a compute node.
- **Or ssh to `login.delta.ncsa.illinois.edu`, run `ws-init`, and paste the `srun` line it prints.**

Getting Started

1 · Launch Code Server

openondemand.delta.ncsa.illinois.edu

Interactive Apps → Code Server

Account	bccu-delta-cpu
Partition	cpu
Reservation	leave empty
CPU's	2
Memory	4 GB
GPU's	0
Duration	1 : 45

Launch → Connect → Terminal, New Terminal

2 · Start the agent, in the terminal

1 `source /work/hdd/bccu/ws-kit/env.sh && ws-init`

2 `cd ~/ws-2026-10-05`

3 `hpc-gpt`

4 `start` when the agent screen appears, then Enter

No Code Server? ssh to `login.delta.ncsa.illinois.edu`, run command 1, and paste the `srun` line it prints.

Harness: `hpc-gpt` is opencode with the NCSA Lumen models. Or start your own in the same folder: `claude`, `codex` or `gemini`.

Finished early? Prompts to keep going

All of these are in `PROMPTS.md` in your workspace. Or type: `Read PROMPTS.md and suggest three things that fit what I have done so far.`

Go deeper on your result

`Explain this result as you would to a new student in my lab, then say what you would check next.`

`Run one more variation that tests <my idea>. Tell me first what you expect.`

`Make another plot from the logs we already have: <what against what>. Save it as .svg.`

`Write what we did as a one-page README I could hand to a colleague.`

Learn about Delta

`Which partitions and GPUs does Delta have, and which would you pick for this job? Use the Delta documentation.`

`Why did my job wait, and how is it charged?`

`How much of my quota am I using, and where should large data go?`

`What would change if I ran this on my own allocation, without the workshop's helpers?`

Bigger steps

All of these are in `PROMPTS.md` in your workspace. Or type: `Read PROMPTS.md and suggest three things that fit what I have done so far.`

A notebook of your results

1 · To the agent:

```
Turn the analysis into a Jupyter notebook in open/ that reads the job logs and
redraws the plots. Use matplotlib.use('svg') and show each figure with
IPython.display.SVG.
```

2 · Then you, in a terminal, once (the module's packages only):

```
module reset && module load pytorch-conda/2.12 &&
setup-jupyter-kernel.py --kernel-name workshop --display-name "Python (workshop)"
```

Or, with the extras you installed into `open/.venv` (a venv on top of a module: a Delta pattern worth knowing). Each command is one line; wrapped here:

```
module reset && module load pytorch-conda/2.12 &&
source open/.venv/bin/activate &&
setup-jupyter-kernel.py --kernel-name workshop-venv
--display-name "Python (workshop + my venv)"
```

3 · Open it: Open OnDemand → Interactive Apps → Jupyter Lab → open the notebook → pick that kernel.

Another prepared topic

A second folder, in a terminal:

```
WS_HOME=~/ws-second ws-init
cd ~/ws-second && hpc-gpt
```

then type `start` and choose another topic

Toward your own project

Read `ON-YOUR-OWN.md` and draft an `AGENTS.md` for my own project: `<two sentences about it>`. Put it in `open/my-project/`.

What are you not allowed to do here, and which file says so?

`/models`, pick another model, and ask it the same question again.

Workshop repo

<https://github.com/ncsa/delta-agents-workshop-kit>